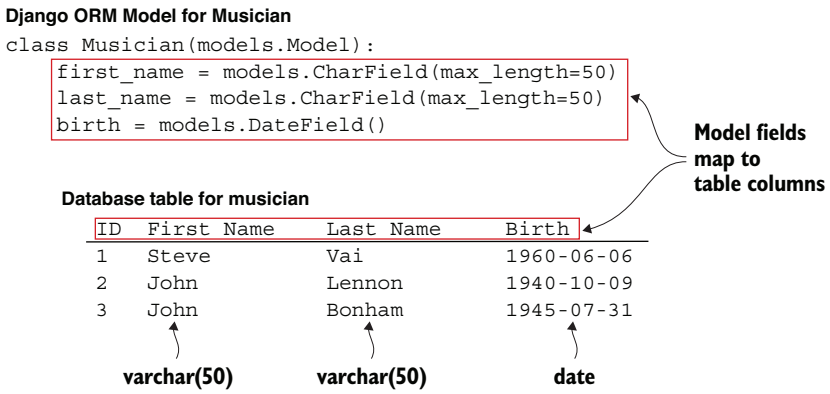


Figure 4.3 `Musicians` ORM `Model` object to database table mapping

The command that creates migration scripts is `makemigrations`. The first time you run this, it creates a migrations directory in the app and an initial migration script. Subsequent calls to the command detect any changes to the model and if found, adds more migration scripts.

Run the `makemigrations` command on the `bands` app using the following command:

```
(venv) RiffMates$ python manage.py makemigrations bands
```

Django responds by telling you what it did—in this case, it created the initial migration script containing a model named `Musician`:

```
Migrations for 'bands':
  bands/migrations/0001_initial.py
    - Create model Musician
```

The migration scripts are Python programs and part of your project like any other code file. The details of these scripts and how and when you might modify them by hand gets covered in a later chapter.

With the initial script in place, you use another management command to perform the migration. The `migrate` command runs any outstanding migration scripts that have not previously run on the database. Under the covers, Django is storing the state of migrations in the database itself, and it knows what scripts have been run and which are new.

Recall from chapter 2 the `18 unapplied migration(s)` warning when first running the development server. Django ships with several apps that help you manage users and the database. These apps themselves have models and corresponding migration scripts. When you first ran the development server, Django was telling you it had detected the presence of migrations that haven't been run. To fix the warning, you